

Snake Species Classification – Capstone Project Proposal

Domain Background: Snake identification is a critical task in wildlife safety and public health. Inexperienced individuals often cannot tell apart a dangerous venomous snake from a harmless one; such mistakes or delays in identification are a major cause of snakebite fatalities¹. For example, two snakes may look very similar in shape and color, with one being highly venomous and the other not¹. Identifying the species correctly is important for **avoiding dangerous encounters** and for administering the right antivenom in case of a bite. However, the **high intra-species variation** (snakes of the same species can look different based on age or region) and **low inter-species variation** (different species can look alike) make this a challenging task for humans¹¹.

Recent advances in computer vision suggest that **deep learning can effectively tackle this identification problem**. Convolutional Neural Network (CNN) models have achieved impressive results in snake classification research. For instance, one study obtained **97% accuracy** in distinguishing 45 snake species using a deep CNN model (VGG16) with image augmentation techniques¹¹. These successes indicate that an automated snake classifier is feasible and could significantly help in both scientific research and improving public safety. **Motivated by these findings and the real-world need**, this project aims to develop a snake species classification system that can be used as a tool to help people avoid encounters with venomous snakes.

Problem Statement: The objective of this project is to **build an image classification system that can identify the species of a snake from a photo**. Given an input image of a snake, the model should output the predicted species (for this project, among five possible species). The problem is a multi-class classification task: each image must be categorized into one of the predefined snake species. This problem is **well-defined and measurable** – we can quantify success by how often the model's predictions match the actual species. In practical terms, the solution will help users determine if a snake pictured is of a venomous species or not, thereby addressing the safety motivation. The scope is limited to five species to ensure focus and achievable results within the capstone timeline, but the approach could later be extended to more species. The key challenge is to achieve high accuracy so that the tool can be reliable; a model that frequently misidentifies snakes would not effectively solve the problem. At least one relevant solution to this problem would be to use a deep learning model (trained on many snake images) that can learn the visual differences between species and make consistent predictions.

Solution Statement: The project proposes a **deep learning-based solution using transfer learning** on a pre-trained CNN model. In particular, a **ResNet-50 architecture** will be fine-tuned on our snake image dataset. ResNet-50 is a 50-layer residual network that has

¹<https://link.springer.com/article/10.1007/s11042-023-16773-0>

demonstrated excellent performance on image recognition tasks and comes pre-trained on ImageNet (a large dataset of general images). By fine-tuning this model on snake images, we leverage learned features like edges, textures, and shapes, adapting them to the specifics of snake species. The solution involves the following components:

- **Model Development:** Fine-tune ResNet-50 on the snake dataset. This means replacing the final layer of ResNet-50 to output 5 classes (the snake species) and training this layer (and possibly a few layers before it) on the new data. The majority of ResNet-50's layers will initially remain frozen (using their pre-trained weights) to act as a fixed feature extractor, especially in early training epochs. Then, some layers may be unfrozen for further fine-tuning. This approach usually yields good results even with limited data, as the model doesn't have to learn from scratch. The output will be a trained model capable of predicting species from images.
- **Web Application:** To make the solution accessible, the trained model will be deployed as a **web application on AWS**. The app will provide an interface where a user can upload a snake photo, and the backend will run the model to classify the snake. We plan to use Amazon Web Services for deployment; for example, using AWS SageMaker or an EC2 instance to host the model, and a simple web framework (like Flask or a lightweight React front-end with API calls) for the user interface. The prediction result (snake species) will be displayed back to the user. By deploying to the cloud, users won't need any special hardware or software beyond a web browser.

This solution is **applicable to the problem and domain** because it directly takes what a person might do (look at a snake photo and guess the species) and automates it using a model trained on thousands of examples. The fine-tuned ResNet-50 can capture subtle visual differences that humans might miss, and the web app makes it practical to use. The entire solution is also **quantifiable and replicable**: we can measure its accuracy on a test set to quantify performance, and because we're using well-known models and cloud services, others can replicate or extend the work using the same dataset and code.

Datasets and Inputs: The dataset for this project comes from a **Kaggle dataset called "Pre-processed Snake Images"**². This dataset contains images of snakes that have been cleaned and standardized for the task of image classification. Key details about the dataset:

- **Origins and Composition:** The dataset was originally part of a Snake Recognition Challenge on Alcrowd and was curated and uploaded to Kaggle by Sameeha Rahman. It includes images of **five snake species**: *Northern Water Snake (Nerodia sipedon)*, *Common Garter Snake (Thamnophis sirtalis)*, *Dekay's Brown Snake (Storeria dekayi)*, *Black Rat Snake (Pantherophis obsoletus)*, and *Western Diamondback Rattlesnake*

²<https://hyper.ai/en/datasets/37144>

(*Crotalus atrox*)². These species provide a good mix for our project, as they include a couple of **venomous snakes (e.g., the rattlesnake)** and **non-venomous snakes**, aligning with the goal of distinguishing dangerous snakes. All images are labeled with the species name.

- **Data Characteristics:** The images have been preprocessed for consistency. According to the dataset description, all images have been resized (or cropped) to **384×384 pixels** and have undergone brightness/contrast adjustments². This preprocessing ensures that each image is usable and that the model training won't be thrown off by extreme variations in image quality or dimension. The total dataset size is about 1.1 GB, indicating there are likely a few thousand images in total (exact counts per class can be explored in the project notebook). The data is expected to be divided into training and testing sets, or we will create such splits. We will reserve a portion of these images (e.g., 15-20%) as a test set that the model will not see during training, to evaluate how well the model generalizes to new snake images.
- **Data Access and Inputs:** We will download the dataset from Kaggle. For training the model on AWS, the images will be uploaded to an Amazon S3 bucket or made available to the AWS SageMaker environment. Each input to the model is an image (which can be fed into the model as a 384×384 pixel RGB image array after appropriate normalization). There is no additional metadata being used — only the image pixels as input and the species label as the output. This is appropriate for the problem since the visual pattern of the snake is the primary feature for identification. If needed, we might perform **data augmentation** on the training images (such as random flips, rotations, or zooms) to bolster the training data and help the model learn invariant features. The dataset as provided is well-suited for the task, and using it ensures that our model development is on solid data ground (rather than trying to scrape together images ourselves, which could be incomplete or biased).

Benchmark Model: To judge the effectiveness of our snake classifier, we will compare it against a couple of baseline models:

- **Pre-trained ResNet-50 (without fine-tuning):** The first baseline is the performance of the unmodified ResNet-50 on our snake dataset. Essentially, we will take ResNet-50 pre-trained on ImageNet and only modify the output layer to have 5 classes, but **not perform any training on the snake data**. This model might be expected to perform only modestly well, since it wasn't specifically trained on snakes (though it might classify some common patterns correctly by coincidence). This baseline will give us a sense of how much improvement our fine-tuning provides. For example, if the raw pre-trained model yields 50% accuracy on the test set, that is a starting point; our fine-tuned model should aim to significantly surpass this. Another simple baseline is **random guessing**, which would yield 20% accuracy on a 5-class problem (or always predicting the most common class in the dataset, which might yield a

slightly higher accuracy if the classes are imbalanced). Our model must do far better than these trivial baselines.

- **Existing Snake Classification Model:** We will also compare our results to at least one reference from academic research. A recent study by Ahmed *et al.* (2024) proposed a deep learning approach for snake species classification and achieved very high accuracy (over 97% on 45 classes) using a CNN model and image augmentation¹¹. While our project deals with fewer classes and a different dataset, this figure provides an upper bound of what is achievable with state-of-the-art methods. Another similar work in the literature (found via ScienceDirect) also explores CNNs for snake identification¹. We will consider the performance reported in such studies as a benchmark for excellence. In practice, our primary benchmark will be the **ResNet-50 before fine-tuning** (to quantify our model's improvement) and possibly a simple model like a logistic regression on top of deep features or a smaller CNN trained from scratch, to illustrate the advantage of transfer learning. Ultimately, success will be determined by comparing our fine-tuned model's accuracy to these baselines. We expect the fine-tuned model to dramatically outperform random or naive baselines and hopefully come close to the performance seen in published studies (aiming for an accuracy in the 85-95% range on our test set).

Evaluation Metrics: For this multi-class classification problem, we will use standard evaluation metrics to quantify model performance:

- **Accuracy:** This is the primary metric – it measures the percentage of snake images correctly classified into the right species. It is calculated as (number of correct predictions) / (total predictions). Accuracy is straightforward to interpret: for example, 90% accuracy means 9 out of 10 snake images are labeled with the correct species by the model. We will use accuracy to compare the overall performance of different models (baseline vs. our model). However, accuracy alone can be misleading if the dataset is imbalanced, so we will look at class-wise performance too.
- **Precision and Recall:** We will compute precision and recall for each snake species class. **Precision** for a class (say, rattlesnake) tells us how many of the images that the model labeled as "rattlesnake" were actually rattlesnakes. **Recall** (sensitivity) for that class tells us how many of the actual rattlesnake images in the dataset the model correctly identified as rattlesnakes. These metrics are important if we care about certain classes more – in our context, one could argue that missing a venomous snake (low recall for the venomous class) or falsely labeling a harmless snake as venomous (precision for venomous class) could both be problematic. We will especially check that the model has high recall for the venomous snake class (so it does not miss identifying a dangerous snake) and high precision (so it doesn't cry wolf too often).

- **F1-Score:** We may also report the F1-score for each class and/or overall. The F1-score is the harmonic mean of precision and recall, giving a single measure that balances the two. This is useful to summarize the model performance per class in one number, especially if we need to compare models that might trade off precision vs recall differently. For a well-performing model on a balanced dataset, precision, recall, and F1 are likely to all be high and in a similar range.

Additionally, we will examine the **confusion matrix** of the model's predictions on the test set. The confusion matrix will show, for each actual species, how the model's predictions are distributed. This can highlight if the model is confusing certain species with each other. All these metrics and tools (accuracy, precision, recall, F1, confusion matrix) will help in assessing whether the model is truly performing well and where it might need improvement. Both the benchmark model and the proposed model will be evaluated with these metrics, to directly compare their performance.

Project Design: The project will be executed in several stages, each corresponding to sections of the proposal:

1. **Data Exploration and Preprocessing:** We will begin with exploring the dataset. This involves loading the images and verifying the dataset structure and labels. We will create training, validation, and test splits (for example, 70% train, 15% validation, 15% test, stratified by species). Basic visualizations will be done to see example images of each snake species and to get an intuition of the differences. We'll also confirm the number of images per class and address any class imbalance if present (if one species has significantly fewer images, we might use augmentation or class weighting during training to compensate). Data preprocessing steps such as normalization (scaling pixel values) will be applied. Also, real-time data augmentation will be set up for training (using transformations like random horizontal flip, small rotations, shifts, etc.) to help the model generalize better.
2. **Model Selection and Setup:** The chosen model architecture is **ResNet-50** (pre-trained on ImageNet). We'll import this model in our deep learning framework (likely PyTorch). We will remove its top layer (which was for ImageNet's 1000 classes) and add a new fully-connected layer (or a small sequence of layers) ending in 5 outputs (one for each snake species), with a softmax activation for classification. Initially, we will freeze the layers of ResNet-50 so that their weights do not change during the first phase of training. This means initially only the new classification layer's weights will be learned. We will compile the model with an appropriate optimizer (for instance, Adam optimizer) and a categorical cross-entropy loss (standard for multi-class classification). We'll also set aside the validation set to monitor performance after each epoch.
3. **Training and Fine-tuning:** We will train the model in two phases. **Phase 1:** Train only the added layer(s) on top of ResNet-50. Because the rest of the network is frozen,

this is essentially training a linear classifier on the features extracted by ResNet. We'll train for a number of epochs (monitoring validation accuracy and loss). We expect validation accuracy to improve and then plateau. **Phase 2:** After the top layer is reasonably trained, we will perform hyperparameter tuning and continue training with a different learning rate and batch sizes. This will select the best learning rate and batch size to fit snake-specific features. This will be tuned based on validation performance. The final model chosen will be the one with the best validation performance.

4. **Evaluation and Validation:** Once training is complete, we will evaluate the final model on the **test set** (the images kept unseen during training and validation). We will compute the metrics described in the Evaluation Metrics section: overall accuracy on the test set, precision and recall for each class, and the F1-score. We will also generate a confusion matrix to visualize the model's errors. These results will be presented in the final report. We will compare these results to the **benchmark models**. For instance, if the baseline (unfine-tuned ResNet) got 50% accuracy and our fine-tuned model achieved 90%, that's a clear improvement. We will also reference the performance of models in literature: if our model gets, say, 90% accuracy, we can note that it's in line with what other research has achieved on similar problems (albeit with different datasets)¹. This analysis will confirm whether the project's solution is meeting expectations and where there is room for further improvement (for example, if one species is still often misclassified, we can note that for future work).
5. **Deployment and Web Application:** After verifying that the model works well, we move to deployment. The trained model will be saved (serialized) and uploaded to an AWS service. Using **AWS SageMaker**, we can create a model endpoint for real-time inference. Alternatively, we might use a Docker container with the model and host it on an AWS EC2 instance or AWS Lambda for serverless execution. Concurrently, a simple **web front-end** will be developed. For example, a Python Flask app or a small JavaScript front-end that allows a user to upload an image file. When an image is uploaded, the front-end will send it to the server (or directly to the SageMaker endpoint via an API call). The model will process the image and return the predicted species label. The web app will then display the result to the user, perhaps along with a confidence score or a statement about the snake (e.g., "Predicted Species: Western Diamondback Rattlesnake – *Venomous*"). During deployment, considerations such as inference latency, handling of large image files, and making the app interface intuitive will be addressed. We will test the deployed model with a few sample images to ensure the end-to-end system works correctly. This stage demonstrates the project's practical usability: the final deliverable isn't just a model in a notebook, but a functional tool that could be used by others (for instance, wildlife enthusiasts or healthcare workers in the field).

Throughout the project, we will document each step, keep track of results (such as training curves and test results), and ensure we meet the criteria laid out in the proposal. The design outlined above ensures that we start from understanding the domain and data, then steadily build up to a working solution, and finally make it accessible as a deployable application. By following this workflow, the project remains organized and each component (data, model, evaluation, deployment) will be addressed thoroughly.

References:

1. Kareem Ahmed, Mai A. Gad, and Amal E. Aboutabl, "*Snake species classification using deep learning techniques*," **Multimedia Tools and Applications**, vol. 83, pp. 35117–35158, 2024. (A study demonstrating deep learning methods for snake classification and highlighting the importance of accurate snake identification)¹¹
2. Sameeha Rahman, *Pre-processed Snake Images* [Dataset]. Kaggle, 2023. Available: [Kaggle.com/datasets/sameeharahman/preprocessed-snake-images](https://kaggle.com/datasets/sameeharahman/preprocessed-snake-images) (A curated dataset of snake images, used in this project for model training and testing)²²
3. Palaniappan, M., et al., "*Deep Learning and Gradient Boosting Ensembles for Classification of Snake Species*," 2024. (Referenced via ScienceDirect: S0041010124003167 – a related model for snake species classification used as a baseline comparison in this proposal).